Genetic Programming for Evolutionary Feature Construction in Symbolic Regression

Hengzhe Zhang

Victoria University of Wellington

September 19, 2025

Outline

1. Introduction
2. Contribution 1: Semantic-Aware Bloat Control
3. Contribution 2: Vicinal Risk Decomposition
4. Contribution 3: Neural-Guided Search
5. Contribution 4: Ensemble Learning
6. Conclusions and Future Work

Regression: The Fundamental Task

Definition:

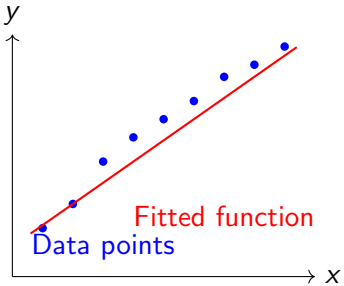
- Predict continuous output Y from input X
- Learn function $f : X \to Y$
- Minimize prediction error

Common Loss Function:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - f(x_i))^2$$

Evaluation Metrics:

- Mean Squared Error (MSE)
- R^2 score: $1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$



Key Challenge: Finding the right functional form

Linear Regression

Model Form:

$$f(x) = \beta_0 + \sum_{j=1}^{m} \beta_j x_j$$

Optimization:

$$\min_{\beta} \|Y - X\beta\|^2$$

Closed-form Solution:

$$\beta = (X^T X)^{-1} X^T Y$$

With Regularization (Ridge):

$$\beta = (X^T X + \lambda I)^{-1} X^T Y$$

Advantages:

- Simple and interpretable
- Fast computation
- Well-understood theory

Limitations:

- Assumes linear relationship
- Cannot capture interactions
- Poor for non-linear patterns

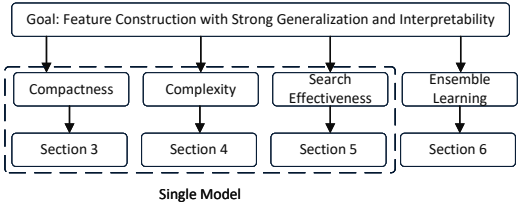
The Feature Construction Challenge

The Problem:

- Raw features X may not capture underlying patterns
- Need to construct $\Phi(X) = \{\phi_1(X), ..., \phi_m(X)\}$
- Balance expressiveness vs interpretability

Why Genetic Programming?

- Variable-length representation
- Building blocks concept
- Gradient-free (handles non-differentiable functions)
- Inherently interpretable



Core Research Questions

How can we enhance GP-based feature construction to achieve both interpretability and generalization?

1. **Bloat:** Can we use semantics to guide tree simplification?
2. **Overfitting:** How to decompose complexity for better control?
3. **Search Efficiency:** Can neural networks accelerate GP search?
4. **Ensemble Power:** How to leverage population-based nature?

Thesis Approach

Address each challenge through complementary innovations, creating a comprehensive framework for evolutionary feature construction

Contribution 1: Semantic-Aware Bloat Control

Bloat in Genetic Programming

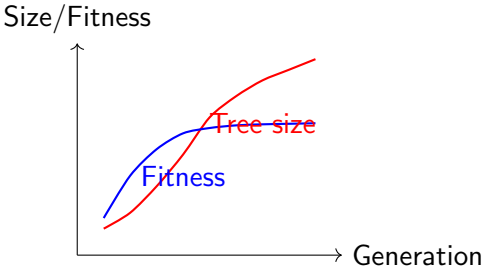
Definition: Growth of program size without corresponding improvement in fitness

Hypotheses for Bloat:

- **Hitchhiking:** Neutral code attaches to fit individuals
- **Defense:** Large trees resist crossover damage
- **Removal bias:** Removing small subtrees less harmful
- **Search space:** Larger programs more common

Consequences:

- Reduced interpretability
- Increased evaluation time
- Memory consumption
- Search inefficiency



Common Control Methods:

- Depth/size limits
- Multi-objective (size vs fitness)
- Selection operators
- Simplification operators

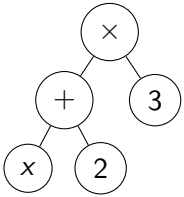
Semantics in Genetic Programming

Syntax vs Semantics:

- **Syntax:** Structure of the program (tree)
- **Semantics:** Behavior/output of the program

Formal Definition:
For a GP tree ϕ and dataset X:

$$\text{Semantics: } s(\phi) = [\phi(x_1), \phi(x_2), ..., \phi(x_n)]$$



Syntax: Tree

Example:

- Tree 1: $(x + 2) \times 3$
- Tree 2: $3x + 6$
- Different syntax, same semantics!

For dataset: $X = \{1, 2, 3\}$

$$s(\phi) = [9, 12, 15]$$

Rethinking Bloat Control: From Random to Semantic

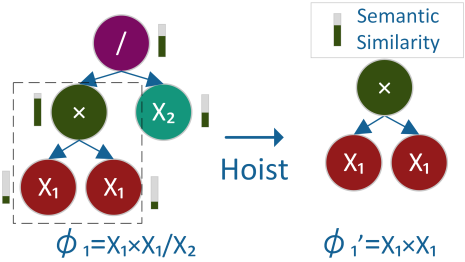
Traditional Approach:

- Random hoist mutation
- Size-based penalties
- No semantic consideration

Our Insight: *Not all subtrees are equal!*

Semantic Hoist Mutation (SHM):

- Measure semantic similarity: $\theta = \cos(\phi(X), Y)$
- Hoist most informative subtree
- Prevent duplicates via hashing



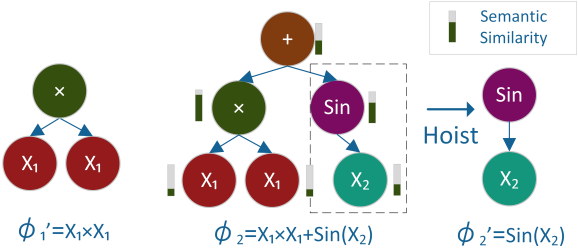
Theoretical Foundation: Why Semantics Matter

Generalization Bound Theorem:

$$L_{exp}(\phi) \leq (1-\theta^2)\left(1-\sqrt{p-p\ln p+\frac{\ln n}{2n}}\right)_{+}^{-1}$$

Key Insight:

- Higher $\theta \rightarrow$ Lower bound
- Semantic similarity directly impacts generalization
- Justifies semantic-based selection



Impact

Trees reduced from ~ 20 nodes to ~ 4.5 nodes

Contribution 2: Vicinal Risk Decomposition

Overfitting in Machine Learning and GP

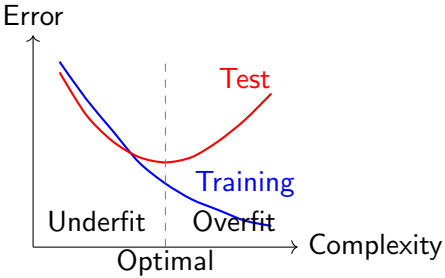
Definition: Model performs well on training data but poorly on unseen data

Causes in GP:

- Complex expressions
- Noise fitting
- Limited training data
- Excessive generations

Detection:

- Training vs test error gap
- Cross-validation performance
- Complexity growth



Control Strategies:

- ✓ Regularization (L1, L2, complexity)
- ✓ Cross-validation
- ✓ Data augmentation
- Ensemble methods

Model Complexity: Size vs Function

Tree Size Complexity:

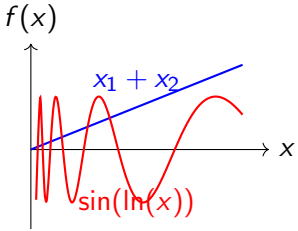
- Number of nodes
- Tree depth
- Number of operations

Functional Complexity:

- Smoothness/roughness
- Number of oscillations
- Sensitivity to input changes
- Curvature

Key Insight:
Size $\neq$ Functional Complexity

Example: Same Size (3 nodes)



Complexity Measures:

- VC dimension
- Rademacher complexity
- Jensen gap
- Lipschitz constant

Mixup: Data Augmentation Technique

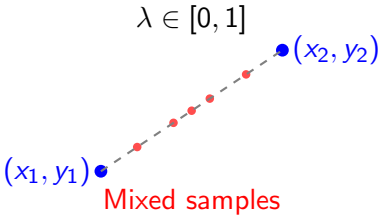
Core Idea: Train on convex combinations of examples

Formula:

$$\tilde{x} = \lambda x_i + (1-\lambda)x_j \quad (1)$$

$$\tilde{y} = \lambda y_i + (1-\lambda)y_j \quad (2)$$

$$\lambda \sim \text{Beta}(\alpha, \alpha) \quad (3)$$



Beta Distribution:

- $\alpha = 1$: Uniform
- $\alpha < 1$: Favor extremes
- $\alpha > 1$: Favor center

Common: $\alpha \in [0.2, 2]$

Benefits:

- Regularization effect
- Smoother regression function
- Better generalization
- Simple to implement

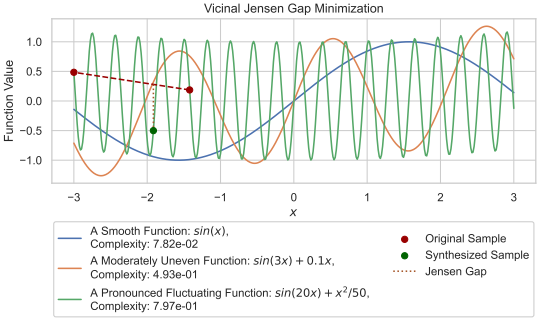
Beyond Model Size: Functional Complexity

Traditional View:

- Overfitting $\leftrightarrow$ Large models
- Control via parsimony pressure

Our Observation:

- $x_1 \times x_2$ vs $\sin(\sin(x))$
- Same size, different complexity!
- Need functional complexity measures



Key Innovation

Decompose vicinal risk into empirical loss + regularization

Vicinal Risk Decomposition: The Theory

Traditional VRM: Minimize loss on synthesized samples

$$\mathcal{V}(f) = \frac{1}{n}\sum_{i=1}^{n}\int \mathcal{L}(f(x), y)\, dP_{x_i}(x)$$

Our Decomposition:

Mixup-based:

$$x_{vic} = \lambda x_i + (1-\lambda)x_j$$

- Empirical loss: O_1
- Vicinal Jensen Gap: O_2
- Promotes local linearity

Perturbation-based:

$$x_{vic} = x_i + \epsilon$$

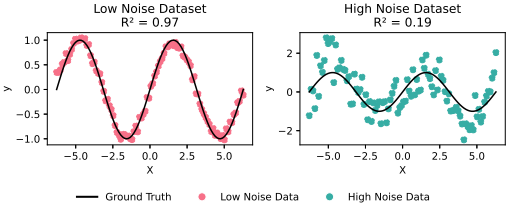
- Empirical loss: O_1
- Finite Difference: O_2
- Controls sensitivity

Why Decompose? Flexible trade-off, accurate training error, adaptable regularization

Smart Regularization: Noise Estimation & Manifold Detection

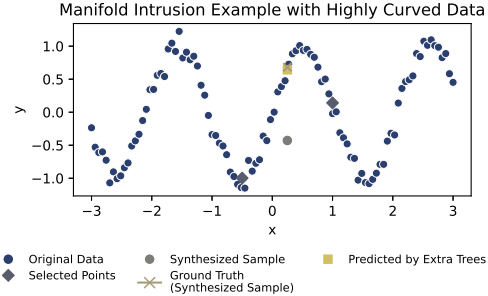
Noise Estimation:

- Use Extra Trees CV score
- Adapt weight τ accordingly
- High noise $\rightarrow$ More regularization

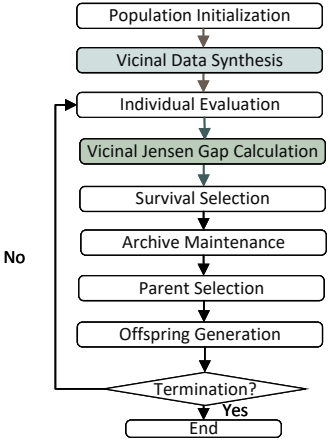


Manifold Intrusion Detection:

- Synthesized data may be unrealistic
- Check with reference model
- Discard intrusion points



VJM-GP: The Complete Framework



Key Components:

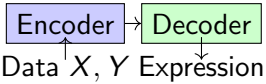
- Multi-objective optimization: O_1 (accuracy) vs O_2 (complexity)
- Lexicase selection for semantic diversity
- Environmental selection via non-dominated sorting
- Adaptive regularization based on dataset characteristics

Contribution 3: Neural-Guided Search

Neural Networks for Symbolic Regression

Neural SR Approaches:

1. **Pre-trained Models:**
   - Train on synthetic datasets
   - Fast inference
   - Limited to training distribution
2. **Reinforcement Learning:**
   - Learn policy for tree generation
   - Flexible function set
   - Sample inefficient
3. **Seq2Seq Models:**



Challenge: Hallucination

- Syntactically correct
- Semantically wrong

Retrieval-Augmented Generation (RAG)

Traditional Generation:

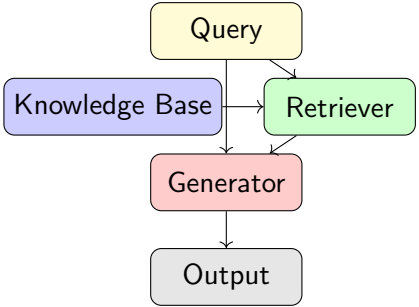
- Code generates from parameters
- Risk of hallucination
- Limited by training data

RAG Framework:

1. **Retrieve:** Find relevant examples
2. **Augment:** Add to context
3. **Generate:** Produce output

Components:

- Knowledge base/library
- Retrieval mechanism (k-NN, dense)
- Generation model



Advantages:

- Reduces hallucination
- Updatable knowledge
- Better grounding
- Interpretable retrieval

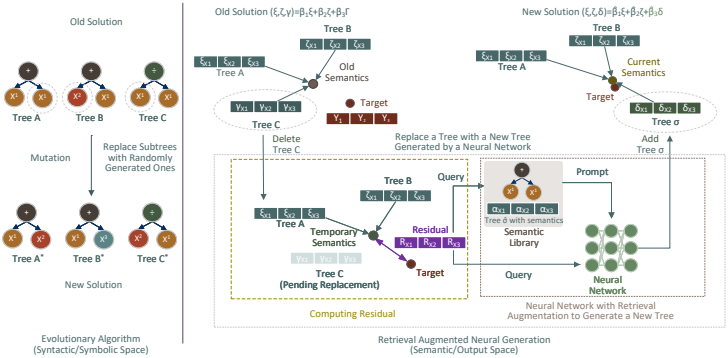
The Knowledge Accumulation Challenge

GP's Limitation:

- No memory across generations
- Inefficient search

Neural Networks' Strength:

- Learn from experience
- Pattern recognition
- BUT: Hallucination problem



Solution: RAG-SR

Combine evolutionary search with retrieval-augmented neural generation

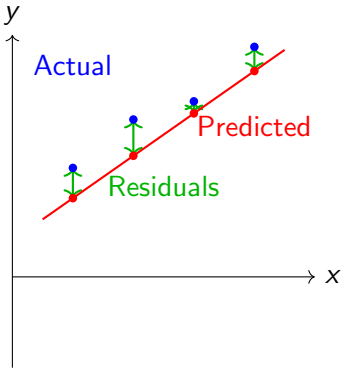
Residuals in Regression

Definition:

$$\text{Residual: } r_i = y_i - \hat{y}_i$$

where:

- y_i = actual value
- $\hat{y}_i$ = predicted value



Semantic Descent: Iterative Refinement

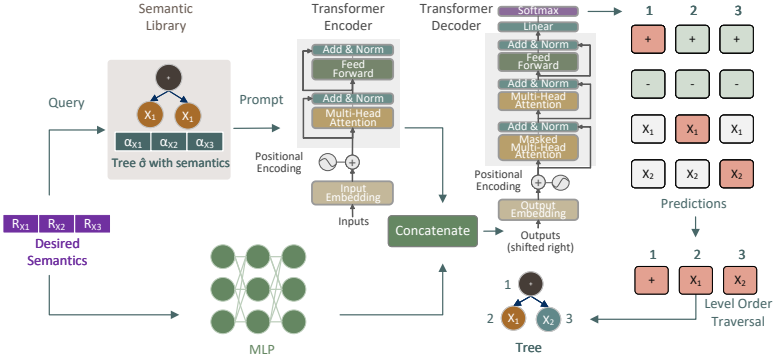
Core Idea: Replace trees to minimize residual
Algorithm:

1. Remove tree ϕ_i contribution
2. Compute residual $R = Y - \Phi^{temp}(X)$
3. Generate replacement:
   - Neural: $\mathcal{N}(R, \mathcal{L})$
   - Retrieval: Best from library
4. Update if improved

Key Innovation:

- **Replace** not add (maintain size)
- Online learning during evolution
- Leverage accumulated knowledge

Retrieval-Augmented Generation Architecture



Three-Component Design:

1. **MLP:** Process desired semantics R
2. **Transformer Encoder:** Encode retrieved tree as prompt
3. **Transformer Decoder:** Generate new symbolic expression

Handling Scale Invariance

Challenge: Linear regression adjusts magnitude automatically

Data Augmentation:

- Include $(\phi, \phi(X))$ and $(\phi, -\phi(X))$
- Sign-insensitive learning

Double Query:

- Query with both R and $-R$
- Select highest probability

Masked Contrastive Loss:

$$\mathcal{L} = \mathcal{L}_{CE} + \lambda \mathcal{L}_{InfoNCE}$$

Aligns semantic embeddings with symbolic representations

Achievement

Online-trained neural SR+GP outperforms pre-trained models

Contribution 4: Ensemble Learning

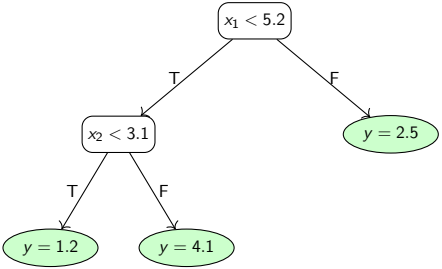
Decision Trees

Structure:

- Binary tree of decisions
- Internal nodes: split conditions
- Leaf nodes: predictions
- Axis-parallel splits

Algorithm (CART):

1. Select best split (MSE/Gini)
2. Partition data
3. Recurse on subsets
4. Stop at criterion



Properties:

- Piecewise constant
- Non-parametric
- Handles non-linearity

Advantages: No scaling needed

Limitations: Overfitting, axis-parallel only

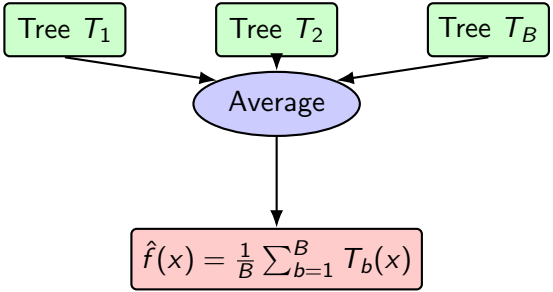
Random Forests: Bias–Variance Tradeoff

Bias–Variance Decomposition:

$$\mathbb{E}\left[(Y - \hat{f}(x))^2\right] = \underbrace{\left(\mathrm{Bias}[\hat{f}(x)]\right)^2}_{\text{systematic error}} + \underbrace{\mathrm{Var}[\hat{f}(x)]}_{\text{sensitivity to data}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

Random Forest intuition:

- Each tree: low bias, high variance
- Averaging many trees $\Rightarrow$ variance drops
- Bias remains low (deep trees, no pruning)
- Net effect: lower test error

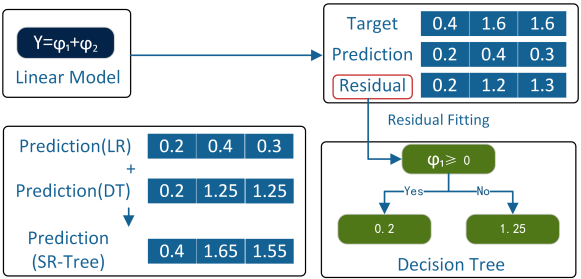


From Individual to Ensemble: SR-Forest

Motivation:

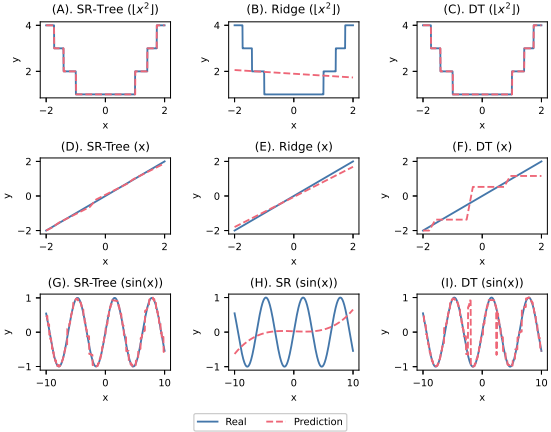
$$E_F = \underbrace{\mathbb{E}[(f-Y)^2]}_{\text{avg loss}} - \underbrace{\mathbb{E}[(f-\bar{F})^2]}_{\text{ambiguity}}$$

- GP is inherently population-based
- Different models capture different patterns
- Heterogeneous strengths



SR-Tree: GP features + Linear (continuous) + DT (piecewise)

Why SR-Tree? Complementary Strengths



Key Insight: Different functions need different models

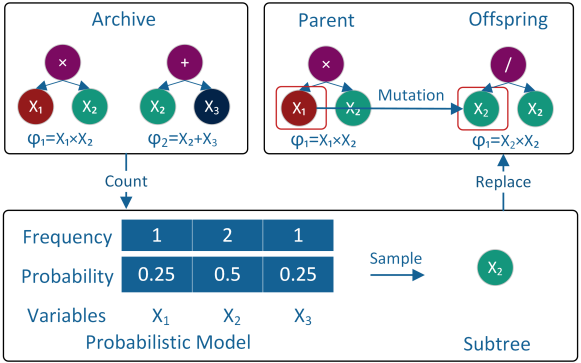
- Linear models: Good for continuous relationships
- Decision trees: Good for piecewise patterns
- Combined: Handle both types effectively

Innovation 1: Guided Mutation for High Dimensions

Problem: Random mutation inefficient with many features

Solution: Probabilistic guidance

- Track feature importance
- Weight by frequency × importance
- Dynamically updated

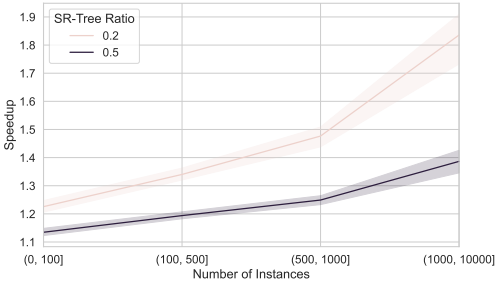


Impact: Significantly better performance on high-dimensional data

Innovations 2 & 3: Efficiency and Selection

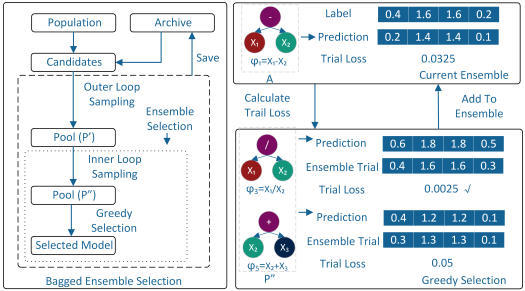
Multi-fidelity Evaluation:

- First 80% generations: Ridge only
- Last 20%: Full SR-Tree
- 1.8× speedup achieved



Ensemble Selection:

- Error reduction-based
- Bagged selection (robustness)
- Semantic diversity preserved

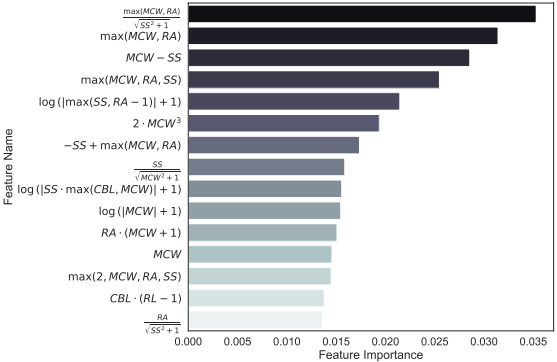


Case Study: Understanding XGBoost

Task: Predict XGBoost performance from hyperparameters

SR-Forest Discovered:

- Min child weight (MCW) crucial
- L1 regularization (RA) important
- Key: Max(MCW, RA)
- Aligns with theory!

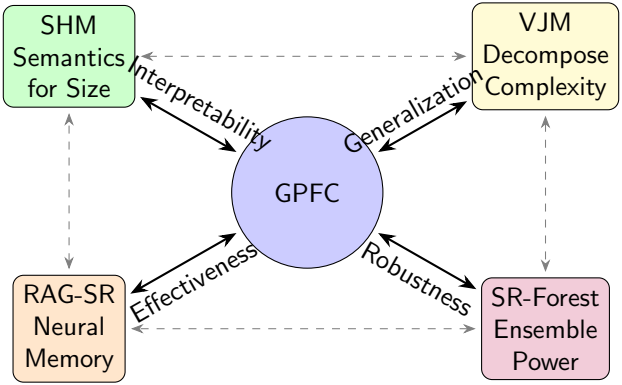


Significance

Data-driven discovery of theoretical relationships

Conclusions and Future Work

A Unified Framework for Feature Construction



Synergistic Effects:

- Each contribution addresses a fundamental challenge
- Can be combined for cumulative benefits
- Forms comprehensive framework for evolutionary feature construction

Summary of Contributions

Challenge	Solution	Key Innovation
Bloat	Semantic Hoist (SHM)	Semantics guide simplification
Overfitting	Vicinal Jensen (VJM)	Decompose complexity
Efficiency	Neural Library (RAG)	Memory + retrieval
Robustness	Ensemble (SR-Forest)	Heterogeneous learning

Publications:

- IEEE TEVC 2024: SHM and SR-Forest
- IEEE TEVC 2025: VJM
- ICLR 2025: RAG-SR

Core Achievement

Advanced GP-based feature construction through complementary innovations addressing interpretability, generalization, efficiency, and robustness

Future Research Directions

Future Research Directions:

- Surrogate models for vicinal risk
- Lightweight transformers (Mamba)
- LLM-driven algorithm design
- Interactive visualization tools
- Industrial deployment frameworks

Ultimate Goal

Bridge symbolic and neural approaches for trustworthy, efficient, interpretable AI

Conclusions

This thesis demonstrates that GP-based feature construction can achieve both interpretability and high performance through:

1. Semantic-aware operations that reduce complexity while improving accuracy
2. Theoretical decompositions that enable flexible overfitting control
3. Neural integration that accelerates search without sacrificing interpretability
4. Ensemble methods that leverage population-based strengths

Questions?

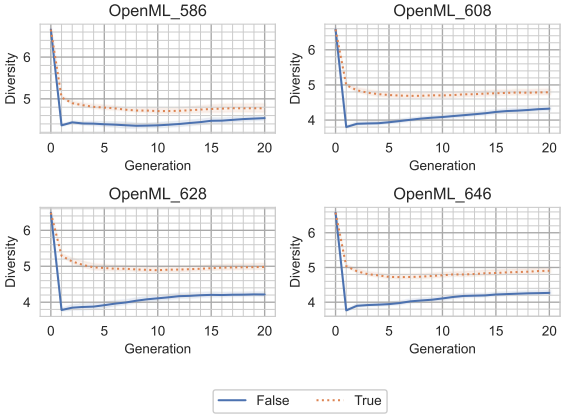
Thank you for your attention!

Maintaining Diversity: Hash-based Checking

Problem: Hoisting best subtree repeatedly → Duplicate features

Solution: Hash-based duplicate prevention

- $O(1)$ checking complexity
- Maintains feature diversity
- Critical for multi-tree GP



Result: Higher phenotype entropy throughout evolution